Laboratório de Concorrência · Aula 10

Vitória Souza Santos · RA 24229424-2 · Turma ESOFT-C
Gerado em 15/10/2026, 22:05:39 · tempo registrado no cronômetro: 36:59

Preparação: Concorrência em cinco minutos

0.1 Com suas palavras: qual é a diferença entre sincronização de cooperação e de competição? Dê um exemplo do dia a dia (fora da computação) para cada uma.

Na cooperação, uma tarefa depende do resultado de outra; por exemplo, numa cozinha, quem monta o prato espera o alimento ficar pronto. Na competição, tarefas disputam um recurso exclusivo; por exemplo, duas pessoas tentando usar a única cabine de atendimento ao mesmo tempo.

Prática 1: A condição de corrida

Previsão: Duas threads somam 1.000.000 cada. Que valor você espera que o programa imprima?

Espero 2.000.000, mas a ausência de sincronização pode fazer o contador terminar abaixo desse valor.

1ª execução (n = 1.000.000)

1012021

2ª execução (n = 1.000.000)

1010137

3ª execução (n = 1.000.000)

1013290

Saída do programa

```
esperado: 2000000
obtido:   1012021

esperado: 2000000
obtido:   1010137

esperado: 2000000
obtido:   1013290

Com n = 1000: obtido 2000 nas duas execuções observadas.
```

1.1 Os valores bateram com o esperado? Foram iguais nas três execuções? Por que mudam de uma execução para outra?

Nas três amostras com um milhão de incrementos, os resultados ficaram abaixo do esperado e variaram. `contador++` envolve ler, somar e gravar; as threads podem intercalar esses passos e sobrescrever atualizações. Com n = 1000, as duas execuções locais deram 2000, mas isso não prova que o programa esteja correto: a intercalação problemática apenas se tornou menos provável.

1.2 Explique, passo a passo, como uma atualização se perde. Use os três passos (ler, somar, gravar) e as duas threads.

Exemplo: contador vale 5. A lê 5; antes de gravar, B também lê 5. A soma 1 e obtém 6; B soma 1 e obtém 6. A grava 6 e B grava 6 por cima. O final é 6 em vez de 7: uma atualização se perdeu.

1.3 Com n = 1000, o que aconteceu? Por que isso torna esse tipo de erro perigoso em sistemas reais? Pense em como o programa se comportaria nos testes e em produção.

Nas duas execuções locais com n = 1000, obtive 2000. Com menos operações, a intercalação que perde incrementos é menos provável, mas ainda possível. Em produção, um erro raro pode passar nos testes e aparecer sob carga, causando dados incorretos intermitentes.

Prática 2: Todas as respostas possíveis (Figura 13.1)

Ordens testadas no simulador

AAABBB → 8

Saída do programa

```
AAABBB → 8
BBBAAA → 7
ABABAB → 6
Ordens possíveis: 20
TOTAL = 4:  9 ordens, por exemplo AABBBA
TOTAL = 6:  9 ordens, por exemplo AABABB
TOTAL = 7:  1 ordens, por exemplo BBBAAA
TOTAL = 8:  1 ordens, por exemplo AAABBB
```

2.1 Quantas ordens são possíveis e quantos resultados diferentes aparecem? Quantas ordens dão o resultado "correto" (8)?

Existem 20 ordens, pois C(6,3) = 20. Surgem 4 resultados distintos: 4, 6, 7 e 8. Apenas 1 ordem dá 8: AAABBB.

2.2 Escolha uma ordem que dá 4 e explique passo a passo por que o resultado é 4. O que aconteceu com a operação de B?

Na ordem AABBBA, A lê 3 e calcula 4; B lê 3, calcula 6 e grava 6; depois A grava seu valor antigo, 4, por cima do 6. O final é 4. A gravação de B foi sobrescrita pela gravação atrasada de A.

Prática 3: Corrigindo a corrida

Previsão: As três versões fazem a mesma conta. Qual você acha que será a mais rápida? E alguma será incorreta?

A versão sem sincronização deve ser a mais rápida, mas pode errar. synchronized e AtomicInteger devem dar 2.000.000; a sincronização tende a custar mais. Os tempos relativos das versões corretas variam conforme a máquina.

Saída do programa

```
esperado:          2000000
sem sincronizacao: 1508299 em 5 ms
synchronized:      2000000 em 133 ms
AtomicInteger:     2000000 em 35 ms
```

3.1 Quais versões deram o valor correto? Compare os tempos: por que a versão correta costuma ser mais lenta que a versão sem sincronização?

Nesta amostra, synchronized e AtomicInteger chegaram a 2.000.000; sem sincronização perdeu incrementos. A versão protegida custa mais porque coordena o acesso concorrente. Aqui synchronized foi mais lenta que AtomicInteger, mas aquecimento da JVM, carga, processador e contenção podem mudar os tempos.

3.2 Uma transferência bancária tira dinheiro de uma conta e coloca em outra. Um AtomicInteger em cada saldo resolve? Por quê? O que seria preciso?

Não. Cada saldo pode ser atualizado atomicamente isoladamente, mas a transferência é uma operação conjunta: debitar e creditar precisam ser indivisíveis para que outra tarefa não observe um estado intermediário ou ocorra apenas uma das alterações. Use uma transação de banco de dados ou uma seção crítica que proteja ambos os saldos, adquirindo locks em ordem consistente.

Prática 4: Impasse (deadlock)

Previsão: Leia o código: A pega X e depois Y; B pega Y e depois X. As mensagens "A terminou" e "B terminou" vão aparecer?

No original, A pode obter X enquanto B obtém Y; cada uma espera o recurso da outra e o detector deve indicar impasse. Assim, não devem aparecer as mensagens de término. Com ambas adquirindo X antes de Y, devem terminar.

Saídas (antes e depois da correção)

```
Antes:
IMPASSE: 2 threads presas
Depois de B também adquirir X e depois Y:
A terminou
B terminou
sem impasse
```

4.1 Por que as duas threads ficaram presas? Descreva o que cada uma tinha e o que cada uma esperava.

A adquiriu X e espera Y; B adquiriu Y e espera X. Cada thread retém um recurso que a outra precisa, e nenhuma o libera antes de adquirir o segundo. Forma-se uma espera circular sem progresso.

4.2 Por que pegar os recursos sempre na mesma ordem elimina o impasse? Dê um exemplo de sistema real em que isso poderia acontecer.

Quando todos adquirem os recursos na mesma ordem global (X antes de Y), não se forma o ciclo em que uma thread retém X e espera Y enquanto outra retém Y e espera X. Em transferências bancárias simultâneas, duas operações podem tentar bloquear as mesmas contas em ordens opostas; ordenar por identificador de conta evita esse ciclo.

Prática 5: Mensagens em vez de memória compartilhada

Previsão: São 6 tarefas de 100 ms divididas entre 3 trabalhadores. Quanto tempo você espera que leve no total? E com 1 trabalhador? E com 6?

Estimativa: 600 ms com 1 trabalhador, 200 ms com 3 e 100 ms com 6, pois as seis tarefas de 100 ms são executadas em sequência, em duas ondas ou numa onda, respectivamente, sem contar sobrecargas.

Tempo com 1 trabalhador

600 ms

Tempo com 3 trabalhadores

200 ms

Tempo com 6 trabalhadores

100 ms

Saída do programa

```
Tempos locais observados:
1 trabalhador, 6 tarefas de 100 ms: levou 600ms
3 trabalhadores, 6 tarefas de 100 ms: levou 200ms
6 trabalhadores, 6 tarefas de 100 ms: levou 100ms
```

5.1 Explique os três tempos. Por que este programa não precisa de synchronized nem de trava?

Com 1 trabalhador, as seis tarefas são sequenciais (~600 ms); com 3, são duas ondas (~200 ms); com 6, uma onda (~100 ms). Não há contador compartilhado: as goroutines recebem tarefas pelo canal. O WaitGroup apenas coordena a conclusão.

Saídas (antes e depois da troca)

```
Sem buffer: fatal error: all goroutines are asleep - deadlock!
Com make(chan int, 1): 1
```

5.2 Por que o primeiro programa trava? Por que o canal com espaço para 1 mensagem resolve? (Dica: num canal sem espaço, quem envia espera alguém receber.)

O canal sem buffer faz o envio esperar até outro goroutine receber. Como a única goroutine tenta enviar antes de receber, ela bloqueia e nunca alcança o recebimento. Com buffer de capacidade 1, a mensagem fica armazenada, o envio termina e o programa pode recebê-la em seguida.

Prática 6: O compilador como guarda

Previsão: Este programa faz uma thread somar 1 a uma variável de main. O que você acha que acontece?

O compilador deve rejeitar o programa porque a thread tenta alterar uma variável local emprestada sem garantir que ela viva o suficiente e que o acesso seja seguro.

Saídas (o erro e o resultado com move)

```
Erro original: Rustc rejeitou com E0373 (closure pode sobreviver à variável emprestada) e
E0502 (conflito de empréstimo mutável e leitura).
Com move: compilou e imprimiu 0.
```

Saída do programa

```
contador = 400000
```

6.1 Com move, o programa compila mas imprime 0. Por quê? (Dica: move faz a thread levar uma cópia do número.)

move transfere o valor para a closure. Como o contador é um inteiro simples, seu valor é copiado para a closure; a thread altera essa cópia, não a variável que permaneceu em main. Por isso main imprime 0. Para compartilhar alterações, o exemplo usa Arc e Mutex.

6.2 Compare com a Prática 1: em que momento o erro foi descoberto em Java e em Rust? Qual é a vantagem e qual é o custo para quem programa?

Na prática 1 em Java, a corrida aparece durante a execução e depende do escalonamento. Em Rust, o exemplo inseguro é barrado na compilação. A vantagem é detectar cedo certas violações de propriedade e compartilhamento mutável; o custo é aprender as regras de propriedade, vida útil e sincronização e estruturar o código conforme elas.

Prática 7: Uma thread só: o laço de eventos

Previsão: Em que ordem você acha que as letras A, B, C e D serão impressas?

A e D primeiro (execução síncrona), depois C (microtask da Promise), depois B (timer); E será impresso após o laço ocupado.

Saída do programa

```
A
D
C
B
E: timer de 10 ms rodou depois de 300 ms
```

7.1 Em que ordem as letras saíram? Por que B, com tempo 0, saiu depois de D e de C?

A e D são síncronos e saem primeiro. Depois que o código atual termina, a microtask da Promise (C) é processada antes dos callbacks de timer. setTimeout com 0 ms não interrompe o código nem executa instantaneamente, então B aparece depois de D e C.

7.2 O timer de 10 ms (linha E) rodou bem depois de 10 ms. Por quê? O que isso significa para um servidor Node.js que atende muitos usuários e faz um cálculo pesado?

O laço ocupado bloqueia a única thread por cerca de 300 ms e impede o event loop de processar callbacks. O prazo de 10 ms é um atraso mínimo aproximado, não uma garantia. Em Node.js, cálculo síncrono pesado bloqueia o atendimento a outros usuários e aumenta a latência; use operações assíncronas, divida o trabalho ou delegue CPU intensiva a workers/processos.

Fechamento: Quem impede a corrida?

Java sem sincronização

Ninguém: há corrida

Java com synchronized

O programador, com sincronização

Go com canais

O projeto: não há memória compartilhada

Rust com Arc e Mutex

O compilador

JavaScript

Só existe uma thread

8.1 Um site de venda de ingressos tem 1 lugar restante, e duas pessoas clicam em Comprar ao mesmo tempo. Qual problema desta atividade pode acontecer? Como você o evitaria?

Pode ocorrer uma corrida e vender duas vezes o último lugar: ambas as requisições leem “1 restante” antes da confirmação. Torne a verificação e decremento atômicos no banco (por exemplo, transação com bloqueio da linha ou UPDATE condicional WHERE lugares > 0) e verifique as linhas alteradas. A garantia deve cobrir todas as instâncias do servidor.

8.2 Em uma frase: o que mais surpreendeu você nesta atividade?

O que mais me surpreendeu foi que contador++, que parece uma operação única, pode perder atualizações porque envolve leitura, cálculo e gravação separados.